

ELETRÔNICA DIGITAL

Maikon Lucian Lenz



SOLUÇÕES
EDUCACIONAIS
INTEGRADAS



Lógica combinacional

Objetivos de aprendizagem

Ao final deste texto, você deve apresentar os seguintes aprendizados:

- Definir portas lógicas.
- Diferenciar teoremas de axiomas.
- Resolver operações de simplificação de expressões booleanas.

Introdução

As portas lógicas são abstrações de circuitos que desenvolvem operações lógicas e possuem simbologia e expressões algébricas próprias que as representam. Essas expressões possibilitam a manipulação algébrica, a fim de obter a saída para determinadas combinações de entradas e, também, a simplificação das expressões para a construção de circuitos digitais reduzidos, capazes de executar a mesma função.

Neste capítulo, você vai estudar as portas lógicas. Você também vai verificar a tabela verdade, que mapeia todas as hipóteses possíveis para um circuito ou expressão lógica, e os métodos utilizados na simplificação por meio de axiomas ou pela técnica gráfica conhecida como mapa de Karnaugh.

Portas lógicas

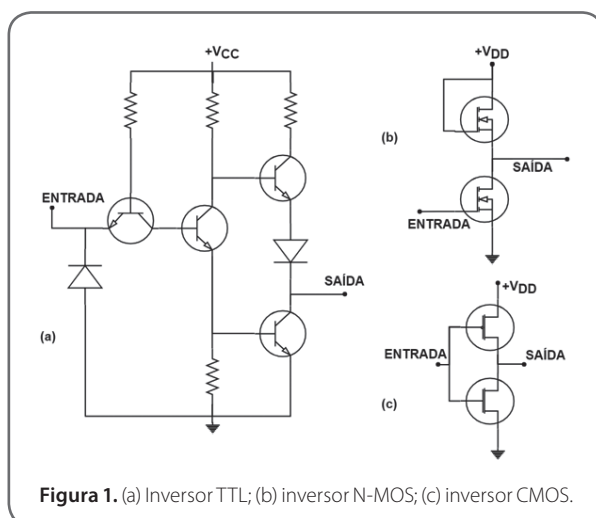
Os estados fundamentais dos circuitos digitais podem ser equiparados a estados lógicos, já que, em ambos os casos, existem dois estados possíveis para uma unidade de informação. Assim, a álgebra booleana, desenvolvida para análise e manipulação de informações de natureza lógica, pode ser utilizada também para sistemas digitais, em que o nível 0 representa o estado falso e o nível 1 representa o estado verdadeiro, conforme lecionam Tocci, Widmer e Moss (2011).

Os circuitos básicos da eletrônica digital são projetados para executar funções lógicas a partir das três operações fundamentais E, OU e NÃO, além de combinações entre estas para a criação de blocos funcionais mais complexos, como somadores, contadores, codificadores, entre outros.

As portas lógicas são criadas com o auxílio de transistores. Geralmente, o transistor é configurado de maneira a operar como uma chave digital — ou seja, transitar entre as regiões de corte e saturação, em vez de operar na região ativa, onde funcionaria como um amplificador. Uma exceção à regra é a tecnologia ECL (lógica com acoplamento pelo emissor), em que a operação ocorre de maneira diferencial, similar ao comportamento de um amplificador operacional, para chavear o sinal a partir da diferença de corrente entre dois ou mais transistores. A vantagem da tecnologia ECL é o tempo de chaveamento, já que os transistores não operam na região de saturação e demandam menos portadores para fazer mudanças de estado, conforme expõe Tocci, Widmer e Moss (2011).

Entretanto, a maior parte dos circuitos lógicos envolve circuitos TTL ou MOS/CMOS. TTL (*transistor-transistor logic*) é a sigla para transistor a transistor e utiliza transistores bipolares polarizados com o auxílio de resistores para desempenhar as funções lógicas, sempre em corte ou saturação. Já MOS é a tecnologia que utiliza transistores de efeito de campo do tipo **metal-óxido-semicondutor** (MOSFET), menores, com potências reduzidas e que dispensam a necessidade de resistores para polarizar adequadamente. Por sua vez, a tecnologia CMOS utiliza os mesmos MOSFETs, porém associados de forma complementar, para que, enquanto um está conduzindo, o outro esteja em corte, o que permite um controle ainda mais eficiente da porta lógica, de acordo com Tocci, Widmer e Moss (2011).

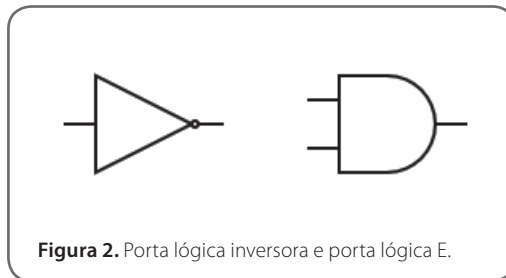
Na Figura 1 é possível ver a comparação de três portas lógicas inversoras utilizando tecnologias diferentes.



É importante compreender o funcionamento a nível elétrico dos circuitos lógicos. Isso permite que se faça o uso mais adequado de cada tecnologia e se compreenda com mais facilidade eventuais defeitos que possam surgir no projeto ou resultantes de circuitos danificados.

Mas, no dia a dia, utilizar o esquema elétrico completo de circuitos digitais não é prático e dificulta a análise fundamental: a mudança de estado de uma saída a partir de um nível de entrada. Para isso, não é necessário observar a atuação de cada componente dentro de uma porta lógica, mas apenas conhecer como a sua saída deve reagir sob condições específicas. Assim, duas formas de abstração são utilizadas para representar um circuito digital de maneira simplificada e que permita utilizar apenas operações lógicas no processo: blocos funcionais e tabela verdade.

O conceito de **blocos funcionais** é fácil de compreender, já que é utilizado por vários outros elementos. Consiste em representar um dispositivo ou um sistema apenas por seus meios de acesso externo, abstraindo toda a informação interna. Nesse tipo de diagrama, há somente pinos e o próprio modelo componente discriminados. Dois exemplos são apresentados na Figura 2.



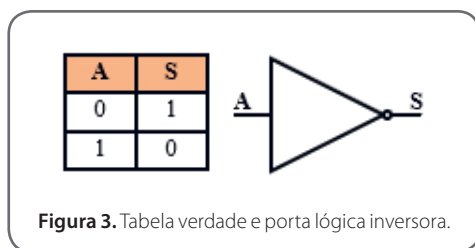
Junto dos blocos, utiliza-se uma tabela com todas as possibilidades lógicas de operação para o determinado circuito. Em outras palavras, é montada uma tabela descrevendo todas as entradas possíveis e os resultados esperados para a saída em cada caso. A **tabela verdade** possui uma coluna para cada entrada ou saída do sistema ou bloco que representa, chamadas de **variáveis**. À esquerda são situadas as variáveis de entrada e, à direita, as de saída. Cada linha representa uma combinação de valores de entrada possível. Normalmente, a tabela é organizada de forma a seguir a sequência numérica natural. Considerando a combinação de entradas, é resolvida a expressão lógica que representa o sistema para cada uma das saídas.

Portas lógicas fundamentais

São três as operações lógicas fundamentais, aquelas que dão origem a todas as outras: E, OU e NÃO. Você vai perceber que o raciocínio por trás delas é muito próximo da nossa forma de pensar.

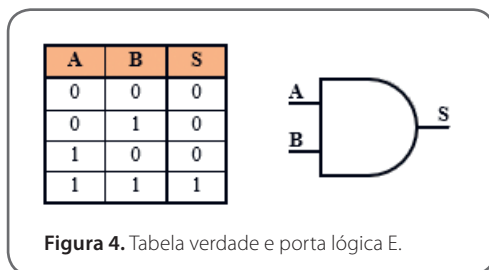
A operação lógica **NÃO**, do inglês NOT, estipula que, para qualquer que seja a entrada, a saída será o inverso. Em outras palavras, se a entrada for falsa, a saída será verdadeira; se a entrada for verdadeira, a saída será falsa. Em eletrônica digital, **falso** é expresso pelo valor 0 e **verdadeiro** pelo valor 1. De agora em diante, serão utilizados 0s e 1s, para facilitar a compreensão e permitir o uso da álgebra booleana mais a frente, conforme sugere Tokheim (2013).

A porta lógica que executa uma operação **NÃO** também é conhecida como **porta inversora**. Essa porta lógica pode ser vista na Figura 3.

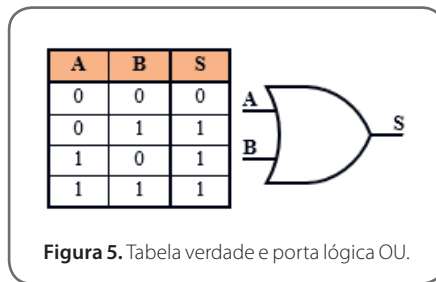


A operação lógica **E**, do inglês AND, diz que: independentemente da quantidade de variáveis de entrada que um sistema possui, o resultado somente será verdadeiro caso todas essas variáveis também o sejam. É o mesmo que dizer, por exemplo, que, para um carro andar, é necessário que esteja ligado E tenha combustível. Repare que, caso qualquer uma das duas condições seja falsa, o carro não entrará em movimento, conforme leciona Tokheim (2013).

A tabela verdade e o símbolo que representa uma porta lógica E estão expressos na Figura 4.



A operação lógica **OU**, do inglês OR, estipula que, independentemente da quantidade de variáveis de entrada, sempre que uma delas for 1, a saída será 1. Em outras palavras, sempre que pelo menos uma delas for verdadeira, a saída será verdadeira; logo, a saída somente será falsa quando todas as entradas forem verdadeiras. Analogamente, utilizando novamente como exemplo um carro, é o que ocorre quando o carro não liga porque está sem bateria OU sem combustível, ou seja, se qualquer das condições for verdadeira (a falta de alguma das condições para ligar o carro), isso fará com que a saída seja verdadeira (o carro esteja com defeito), conforme aponta Tokheim (2013). A Figura 5 apresenta a tabela verdade e a porta lógica OU.



Esse último exemplo é bastante pertinente, uma vez que faz um raciocínio aparentemente contrário ao que seria natural. É comum associarmos coisas positivas com uma resposta verdadeira ou entradas verdadeiras. No caso anterior, a afirmação seguia no sentido inverso — a condição verdadeira representava uma falha. Não que a lógica tenha mudado, mas o significado que se dá para cada evento e o que se interpreta do resultado é que seguiu em um sentido não tão prático quanto poderia. É bom que o leitor perceba a diferença entre variável e fenômeno associado — não há qualquer juízo de valor em uma operação lógica. Em outras palavras, a álgebra utilizada para solucionar problemas lógicos não se importa com o significado dado a cada sinal ou resultado, mas apenas com as regras adotadas para o sistema.

O exemplo do carro que não liga poderia ser reorganizado utilizando uma operação lógica E, sem qualquer prejuízo para o sistema. Por exemplo: o carro liga se houver combustível E houver bateria. O que parece muito mais prático de interpretar.

É importante tentar ser o mais claro possível na forma de lidar com cada variável. Repare que, no primeiro caso, deveria existir um sensor conectado à bateria, que fosse acionado quando a bateria acabasse, e outro que fosse acionado quando o combustível acabasse, para que a operação OU indicasse

a falta de qualquer um deles. Já na segunda situação, os sensores deveriam indicar o inverso, a existência de combustível e de bateria, para que a operação E sinalizasse que está tudo certo para ligar o carro. Perceba o quão importante é o significado atribuído a cada sinal. Errar nessa construção ou na forma de interpretar as informações presentes no sistema fará a diferença entre o sucesso e o fracasso do projeto.

Matematicamente, cada porta lógica fundamental é representada por um símbolo. A operação E é apresentada por um “ $\cdot$ ”, a OU é representada pelo símbolo “ $+$ ”, e a inversora é representada por um traço acima da variável ou conjunto de operações a ser invertida. A prevalência é dada à operação NÃO, quando relacionada a uma única variável, e à operação E, quando entre duas variáveis. As expressões que representam as operações fundamentais podem ser vistas no Quadro 1.

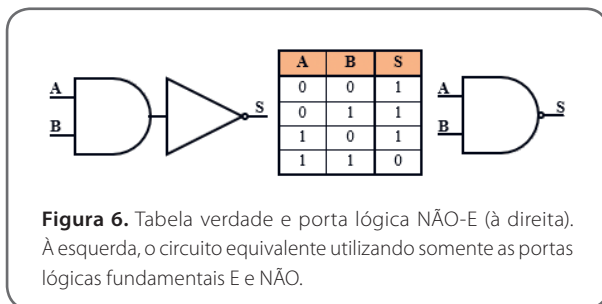
Quadro 1. Expressões booleanas das operações lógicas fundamentais

Operação E	$S = A \cdot B$
Operação OU	$S = A + B$
Operação NÃO	$S = \bar{A}$

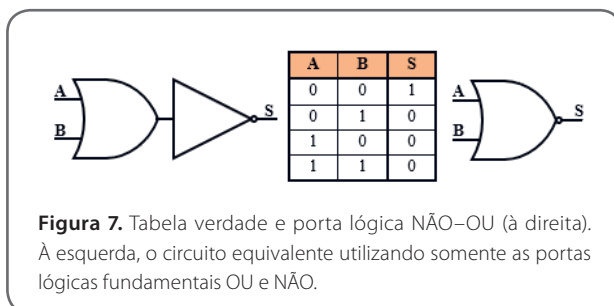
Outras portas lógicas

As portas lógicas fundamentais podem ser associadas para formar outra, mas com tabela verdade diferente. São utilizadas, ao todo, sete portas lógicas diferentes, sendo três delas as portas fundamentais, já apresentadas, e mais quatro resultantes de alguma combinação de maior uso entre elas, que abrangem as portas NÃO-E, NÃO-OU, OU-EXCLUSIVO e NÃO-OU-EXCLUSIVO.

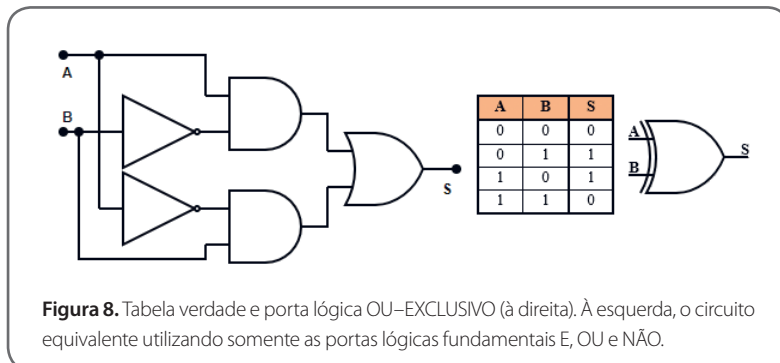
A porta lógica **NÃO-E**, do inglês NAND, é a negação da função E, ou seja, é equivalente a utilizar uma porta inversora em série com uma porta E, conforme aponta Tokheim (2013). O resultado será a inversão de todos os casos da tabela verdade de uma operação lógica E (Figura 6).



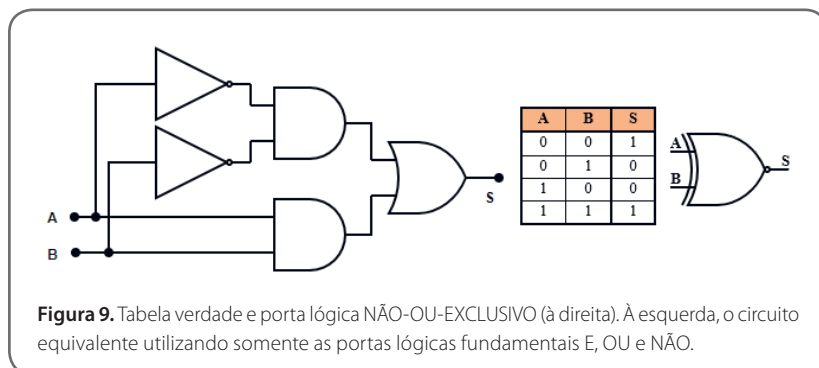
A porta lógica **NÃO-OU**, do inglês NOR, é a negação da função OU, ou seja, é equivalente a utilizar uma porta inversora em série com uma porta OU, segundo Tokheim (2013). O resultado será a inversão de todos os casos da tabela verdade de uma operação lógica OU (Figura 7).



A porta lógica **OU-EXCLUSIVO**, do inglês XOR, apresenta saída 1 quando apenas uma das entradas for verdadeira. Para construí-la, é necessário utilizar cinco portas lógicas fundamentais, conforme leciona Tokheim (2013) (Figura 8).



A porta lógica **NÃO-OU-EXCLUSIVO**, do inglês XNOR, apresenta saída 1 quando ambas as entradas forem verdadeiras. Para construí-la, também é necessário utilizar cinco portas lógicas fundamentais, segundo Tokheim (2013). Essa porta lógica é conhecida como **coincidência**, já que, somente quando todos os dados da entrada coincidirem (sejam todos 0s ou 1s), a saída será verdadeira (Figura 9).



Assim como para as portas lógicas fundamentais, as demais também podem ser expressas matematicamente, conforme mostra o Quadro 2.

Quadro 2. Expressões booleanas das operações lógicas NAND, NOR, XOR e XNOR

Operação NÃO-E	$S = \overline{A \cdot B}$
Operação NÃO-OU	$S = \overline{A + B}$
Operação OU-EXCLUSIVO	$S = \overline{A} \cdot B + A \cdot \overline{B}$ ou $S = A \oplus B$
Operação NÃO-OU-EXCLUSIVO	$S = \overline{A} \cdot \overline{B} + A \cdot B$ ou $S = A \odot B$

Obtenção da expressão lógica correspondente

A tabela verdade é o mapeamento de cada resposta possível de um sistema digital. A partir dela, pode-se obter as expressões booleanas que simplificam a análise e a visualização das conexões existentes entre as portas lógicas. Para

obter a expressão lógica a partir de uma tabela verdade, pode-se utilizar duas técnicas: soma dos produtos e produto das somas.

Na soma dos produtos, serão consideradas apenas as linhas (combinações de entradas) cuja saída resulte em 1. Cabe ressaltar, primeiramente, que, para cada saída (coluna), existe uma expressão lógica independente das demais. O procedimento consiste em fazer a operação E entre todas as entradas de cada linha igual a 1, considerando o inverso das variáveis, cuja entrada para a linha em questão seja 0. Ao final, as expressões obtidas para cada linha utilizada deverão ser agrupadas por uma operação OU.

Considere a tabela verdade da Figura 9, em que S é igual a 1 apenas nas linhas 1 e 4. Na linha 1, ambas as variáveis, A e B, são iguais a 0. Nesse caso, deve-se fazer uma operação E entre todas as entradas; por serem iguais a zero, todas estarão barradas ou sinalizadas por um apóstrofo. Ou seja, cada variável deverá passar por uma operação NÃO antes de ser vinculada à operação E. O resultado da linha 1 é:

$$S_1 = \bar{A} \cdot \bar{B} \text{ ou } S_1 = A' \cdot B'$$

Para a linha 4, A e B são verdadeiros, e ainda passarão por uma operação E; porém, não serão invertidas antes da operação:

$$S_4 = A \cdot B$$

Uma vez obtidas as expressões para cada hipótese (linha) em que o sistema possui saída verdadeira, todas as hipóteses devem ser atribuídas a uma operação OU:

$$S = S_1 + S_4$$

$$S = \bar{A} \cdot \bar{B} + A \cdot B$$

Trata-se da mesma expressão já demonstrada na tabela anterior para a operação lógica *XNOR*.

Já no produto das somas, inverte-se toda a lógica, e passam a ser consideradas apenas as saídas iguais a 0. No lugar da operação E, as entradas são relacionadas por uma operação OU, e são invertidas as entradas que apresentem valor 1. Ao final, as expressões obtidas são agrupadas por uma porta E. Todos os procedimentos são invertidos, já que estão sendo buscadas saídas falsas em vez de verdadeiras.

Utilizando novamente a tabela verdade da Figura 9, observa-se que as linhas 2 e 3 apresentam saídas iguais a 0. Na linha 2, a entrada A é 0 e deve ser mantida, enquanto a entrada B é 1 e deve ser invertida; só então é que serão operadas pela lógica OU:

$$S_2 = A + \bar{B}$$

Na linha 3, a situação se inverte; a saída A é verdadeira, enquanto a saída B é falsa. Logo, quem deverá passar por uma porta NÃO antes da porta OU é a entrada A:

$$S_3 = \bar{A} + B$$

Como a operação E tem prevalência sobre a operação OU, ao agrupar as duas expressões, deve-se utilizar parênteses para que fique explícita a necessidade de resolver as funções OU antes das funções E.

$$S = S_2 \cdot S_3$$

$$S = (A + \bar{B}) \cdot (\bar{A} + B)$$

Se resolvidas caso a caso, o resultado de cada expressão obtida deverá ser a mesma tabela verdade (a mesma resposta para todas as hipóteses), seja pela soma dos produtos, seja pelo produto das somas. O resultado pode ainda ser expandido, resolvendo-se as “multiplicações” algébricas.

$$S = A \cdot \bar{A} + A \cdot B + \bar{A} \cdot \bar{B} + B \cdot \bar{B}$$

Você vai aprender sobre teoremas e axiomas booleanos mais adiante, mas é possível provar que a expressão expandida é a mesma obtida pela soma dos produtos utilizando apenas um deles. Se cada variável só pode possuir dois estados válidos, 0 ou 1, e estão presentes operações E entre a variável e o seu próprio inverso, como em $A \cdot \bar{A}$ e $B \cdot \bar{B}$, é fácil demonstrar que qualquer que seja o valor de A ou B, o resultado dessas operações E será sempre 0.

$$S = 0 + A \cdot B + \bar{A} \cdot \bar{B} + 0$$

$$S = \bar{A} \cdot \bar{B} + A \cdot B$$

Resta claro que, qualquer que seja o método de obtenção das expressões a partir da tabela verdade, o resultado deverá ser o mesmo, independentemente das expressões obtidas. As portas lógicas e suas ligações podem ser desenhadas seguindo a ordem em que cada operador aparece, desde que sejam obedecidos os critérios de equivalência já citados. Ao final de um circuito completo, a última saída deverá representar a variável à esquerda da equação; no caso dos exemplos anteriores, S.

Teoremas e axiomas

Os teoremas e axiomas são parte de um conjunto de regras utilizadas para simplificar as expressões lógicas, conforme leciona Vahid (2008). Os axiomas mais importantes são apresentados no Quadro 3.

Quadro 3. Expressões booleanas das operações lógicas fundamentais

Intersecção	$A \cdot 1 = A$	$A \cdot 0 = 0$
União	$A + 1 = 1$	$A + 0 = A$
Tautologia	$A \cdot A = A$	$A + A = A$
Comutativa	$A \cdot B = B \cdot A$	$A + B = B + A$
Associativa	$(A \cdot B) \cdot C = A \cdot (B \cdot C) = A \cdot B \cdot C$	$A + (B + C) = (A + B) + C = A + B + C$
Distributiva	$A \cdot (B + C) = A \cdot B + A \cdot C$	$A + B \cdot C = (A + B) \cdot (A + C)$
Absorção	$A + A \cdot B = A$	$A \cdot (A + B) = A$
Dupla negação	$\bar{\bar{A}} = A$	
Complementares	$A \cdot \bar{A} = 0$	$A + \bar{A} = 1$

O **teorema de Morgan**, por sua vez, estabelece uma relação de dualidade entre somas (OU) e produtos (E) e afirma que o inverso de um produto é equivalente à soma das entradas invertidas:

$$\overline{A \cdot B} = \bar{A} + \bar{B}$$

O contrário também é verdadeiro:

$$\overline{A + B} = \bar{A} \cdot \bar{B}$$

A partir dele, é ainda mais fácil perceber que a soma dos produtos e o produto das somas representam o complemento de uma mesma tabela verdade; ou seja, enquanto uma estabelece as regras para se obter as saídas 0, a outra estabelece as regras para se obter as saídas iguais a 1. De qualquer forma, ambas chegarão ao mesmo resultado, como já foi demonstrado no exemplo da porta XNOR.

Simplificação de expressões booleanas

Simplificar uma expressão booleana consiste em reduzir a quantidade de operações lógicas necessárias e de variáveis utilizadas, sem que seja afetado o resultado final para qualquer uma das possibilidades que envolvem o circuito. A simplificação pode ser feita diretamente por meio das expressões booleanas, com o auxílio dos teoremas e axiomas apresentados. Buscam-se partes de uma expressão que atendam aos critérios dos axiomas ou teoremas, para que sejam substituídas por uma quantidade menor de operações e/ou variáveis. Um exemplo de simplificação utilizando o axioma complementar já foi realizado ainda na simplificação da equação para a porta lógica XNOR.

Porém, quanto maior for a expressão, maior será o trabalho de simplificação algébrica, e, muitas vezes, você poderá se deparar com diferentes caminhos, que nem sempre o levarão para o ponto de máxima redução. Para facilitar o procedimento, Maurice Karnaugh desenvolveu um método de solução gráfica para obter as expressões já simplificadas, o qual batizou de **mapa de Karnaugh**, conforme leciona Tokheim (2013).

O mapa é útil para tabelas verdade que possuam até seis entradas, e não há limites para a quantidade de saídas. Basta que, para cada saída, seja desenvolvido um mapa, já que cada uma delas se refere a uma expressão individual. Teoricamente, o mapa poderia ser solucionado com mais de seis entradas, mas, por se tratar de um método gráfico, a complexidade aumenta consideravelmente a cada nova entrada.

O mapa de Karnaugh reorganiza as **linhas de hipóteses** para um modelo bidimensional. As combinações de entrada não serão mais o conjunto de valores de uma única linha, mas o cruzamento de uma linha com uma coluna, com intersecção em uma célula de saída. A Figura 10 mostra a conversão de uma tabela verdade de duas entradas para um Mapa de Karnaugh.

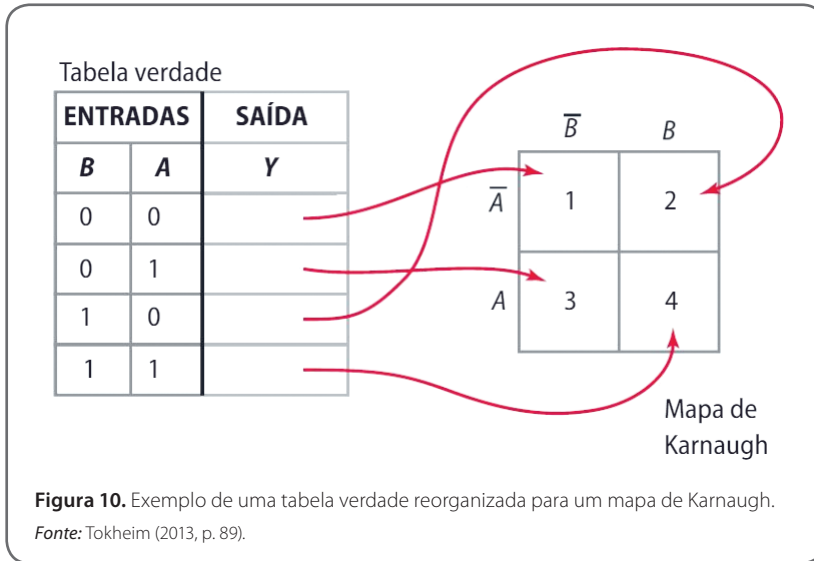


Figura 10. Exemplo de uma tabela verdade reorganizada para um mapa de Karnaugh.

Fonte: Tokheim (2013, p. 89).

No exemplo da Figura 10, as linhas e colunas só dependem de uma variável cada. Mas, para casos com três ou mais variáveis, deve-se obedecer ao **critério da adjacência**, em que a combinação de valores entre colunas ou linhas vizinhas só pode conter uma única mudança de estado. Em outras palavras, de uma linha ou coluna para a próxima, somente o estado de uma entrada pode sofrer alteração por vez. Isso significa que a ordem numérica não será mais obedecida.

Considere, por exemplo, a segunda linha de combinações de uma tabela verdade com três variáveis de entrada. Seguindo a ordem natural, esta teria as seguintes hipóteses de entrada 001_2 , a terceira linha por sua vez seria 010_2 , ou seja, o primeiro bit mudou de 1 para 0, o segundo, de 0 para 1, e somente o terceiro permaneceu em 0. Nesse caso, se as três variáveis fossem distribuídas em uma matriz de linha A e colunas B e C, a ordem de colunas passaria de $00_2 \rightarrow 01_2 \rightarrow 10_2 \rightarrow 11_2$ para $00_2 \rightarrow 01_2 \rightarrow 11_2 \rightarrow 10_2$, de forma que a transição da segunda para a terceira coluna tenha apenas uma mudança de estado. Todas as hipóteses continuam presentes, já que apenas a ordem de transição foi alterada. A linha A, com as condições 0_2 e 1_2 apenas, não necessitaria de qualquer intervenção, já que uma única variável atende intrinsecamente ao critério de adjacência, conforme leciona Floyd (2007).



Fique atento

Ao mudar a sequência em que aparecem as hipóteses no mapa de Karnaugh, para cumprir o critério de adjacência, deve-se ter o cuidado para que as saídas continuem relacionadas à hipótese específica. Em outras palavras, se uma coluna ou linha é alterada de posição, as células da saída correspondentes àquele valor devem seguir com ela. Isso porque a resposta de uma tabela verdade não está associada a uma posição específica, mas à combinação de valores das entradas, e as combinações devem ser mantidas para não alterar o significado e o resultado do sistema.

Uma vez construído o mapa de Karnaugh, são feitos enlaces, que devem agrupar somente células vizinhas, vertical ou horizontalmente, que possuam valor 1. O tamanho de um enlace é dado em potências de 2, podendo cobrir uma, duas, quatro ou oito células e, assim, sucessivamente. Os enlaces devem ser quadrados ou retangulares e jamais desenhos e/ou contornos estranhos a esse formato. Isso porque o que se faz graficamente pretende simplificar a expressão, que, se feita manualmente, atenderia à correspondência algébrica necessária; qualquer enlace que não atenda as regras não teria equivalência matemática e, portanto, adicionaria um erro à expressão.

A exceção aos desenhos quadrados/retangulares deve ser feita para as extremidades, em que é possível agrupar as células, pois a matriz deve ser vista como uma esfera fechada, em que as extremidades estão unidas. Recorde-se que o critério de adjacência estabelece que duas células são ditas vizinhas se suas linhas e colunas possuem apenas uma transição de estado entre elas, o que não deixa de ser verdade para as células das extremidades. Quando um enlace não for suficiente para, atendendo às regras anteriores, cobrir todas as células iguais a 1, mais enlaces devem ser formados, até que o processo esteja completo.

Os enlaces podem se sobrepor, e cada célula pode pertencer a mais de um enlace, o que, de um modo geral, aumenta a simplificação, por possibilitar enlaces maiores. Porém, deve-se tomar cuidado para que não sejam criados enlaces desnecessários, que aumentariam a expressão, sem adicionar qualquer outra célula igual a 1 descoberta. Em raras situações, os enlaces podem necessitar de procedimentos mais rígidos para se escolher o tamanho dos enlaces e quais células cobrir primeiramente. Mas, de um modo geral, é possível preencher o mapa de Karnaugh sem problemas, buscando os maiores enlaces possíveis e reduzindo o tamanho do enlace enquanto ainda existirem células de valor 1 descobertas, conforme leciona Floyd (2007).

Cada um dos enlaces representa uma operação E da expressão, e as várias expressões/enlaces são somadas (operação OU) para dar origem à expressão completa daquela saída para a qual se construiu o mapa. Um a um, os enlaces devem fazer o produto (operação E) das entradas cujos estados vinculados às suas células não tenham mudanças de valor.



Saiba mais

Cada enlace do mapa de Karnaugh é capaz de eliminar as condições atendidas pelos axiomas complementares, em que uma expressão conteria situações como $A + \bar{A}$. Apesar de um mapa de Karnaugh conter, muitas vezes, diversas soluções, se atendidas as regras estipuladas, todas elas devem obter o mesmo nível de simplificação, mesmo que as expressões não sejam coincidentes.

Se um enlace cobre as hipóteses em que A é igual a 0 e 1, mas somente as hipóteses em que B e C são iguais a 10_2 , a variável A deverá ser descartada para a expressão desse enlace, já que muda de valor ao longo dele; como B é sempre 1 e C é sempre 0, devem ser utilizados na expressão. Considere as variáveis que mantenham o valor 1 de forma direta na expressão, e as variáveis iguais a 0 de forma inversa. Ou seja, sendo $BC = 10_2$ para todas as células do enlace, a expressão correspondente a ele será $B \cdot \bar{C}$. Lembrando, mais uma vez, que A foi eliminado da expressão porque não possui um valor fixo para todas as células do enlace analisado. Uma vez obtidas as expressões equivalentes a cada enlace, a expressão final será a soma de todas elas.

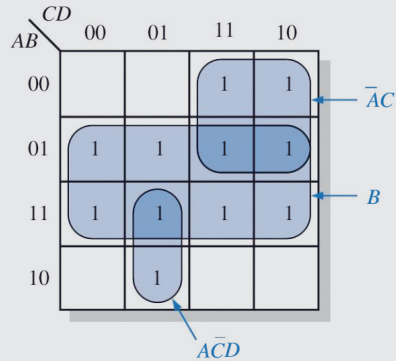


Exemplo

Considere o mapeamento das possibilidades para um circuito digital representado pela tabela verdade a seguir. Qual seria a expressão mais simplificada?

A	B	C	D	S
0	0	0	0	0
0	0	0	1	0
0	0	1	0	1
0	0	1	1	1
0	1	0	0	1
0	1	0	1	1
0	1	1	0	1
0	1	1	1	1
1	0	0	0	0
1	0	0	1	1
1	0	1	0	0
1	0	1	1	0
1	1	0	0	1
1	1	0	1	1
1	1	1	0	1
1	1	1	1	1

O mapa de Karnaugh para o exemplo ficaria assim (as células iguais a 0 foram deixadas sem valor, já que não são utilizadas):



Fonte: Floyd (2007, p. 233).

Repare que, quanto maior o enlace, menor a quantidade de variáveis da expressão. O enlace maior, de oito células, engloba as linhas 2 e 3, em que a variável A muda de valor e fica de fora, mas B é sempre 1. Para as variáveis das colunas, todas mudam de valor e são eliminadas da expressão.

$$S_1 = B$$

No enlace de quatro células, B deve ser eliminado, já que nas linhas 1 e 2 ele muda de valor. Para as colunas, apenas C mantém o valor sempre em 1, sendo eliminada a variável D. A variável é invertida porque, apesar de manter seu valor dentro do enlace, ela é igual a 0.

$$S_2 = \bar{A} \cdot C$$

O último enlace possui apenas duas células, e apenas a variável B muda de valor ao longo dele. Sendo todas as demais permanentes e iguais a 1, a expressão será:

$$S_3 = A \cdot C \cdot D$$

A expressão simplificada resulta da soma das expressões de cada enlace.

$$S = S_1 + S_2 + S_3$$

$$S = B + \bar{A} \cdot C + A \cdot C \cdot D$$



Referências

FLOYD, T. L. *Sistemas digitais: fundamentos e aplicações*. 9. ed. Porto Alegre: Bookman, 2007.

TOCCI, R. J.; WIDMER, N. S.; MOSS, G. L. *Sistemas digitais: princípios e aplicações*. 11. ed. Rio de Janeiro: Pearson, 2011.

TOKHEIM, R. *Fundamentos de eletrônica digital: sistemas combinacionais*. 7. ed. Porto Alegre: AMGH, 2013.

VAHID, F. *Sistemas digitais: projeto, otimização e HDLs*. Porto Alegre: Bookman, 2008.

Leituras recomendadas

CAPUANO, F. G.; IDOETA, I. V. *Elementos de eletrônica digital*. 41. ed. São Paulo: Érica, 1997.

SZAJNBERG, M. *Eletrônica digital: teoria, componentes e aplicações*. Rio de Janeiro: LTC, 2014.

TOKHEIM, R. *Fundamentos de eletrônica digital: sistemas sequenciais*. 7. ed. Porto Alegre: AMGH, 2013.

Encerra aqui o trecho do livro disponibilizado para esta Unidade de Aprendizagem. Na Biblioteca Virtual da Instituição, você encontra a obra na íntegra.

Conteúdo:



SOLUÇÕES
EDUCACIONAIS
INTEGRADAS